

瑞吉外卖-第五天

课程内容

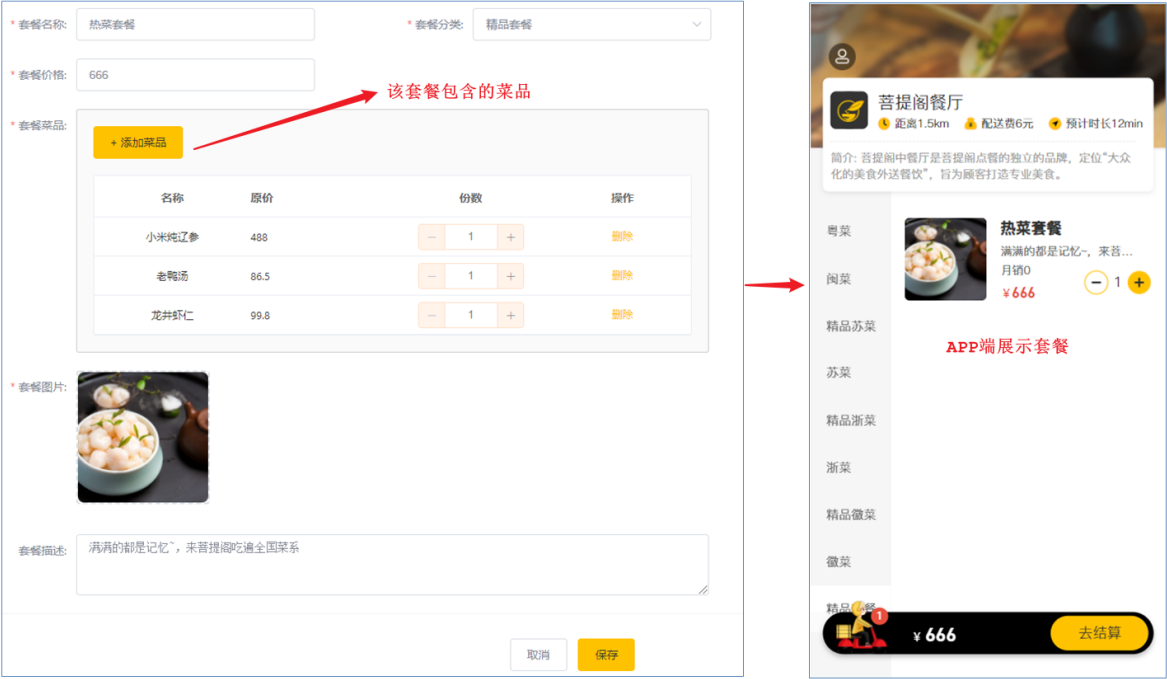
- 新增套餐
- 套餐分页查询
- 删除套餐
- 短信发送
- 手机验证码登录

1. 新增套餐

1.1 需求分析

套餐就是菜品的集合。

后台系统中可以管理套餐信息，通过新增套餐功能来添加一个新的套餐，在添加套餐时需要选择当前套餐所属的套餐分类和包含的菜品，并且需要上传套餐对应的图片，在移动端会按照套餐分类来展示对应的套餐。



1.2 数据模型

新增套餐，其实就是将新增页面录入的套餐信息插入到setmeal表，还需要向setmeal_dish表插入套餐和菜品关联数据。所以在新增套餐时，涉及到两个表：

表	说明	备注
setmeal	套餐表	存储套餐的基本信息
setmeal_dish	套餐菜品关系表	存储套餐关联的菜品的信息（一个套餐可以关联多个菜品）

两张表具体的表结构如下：

1) . 套餐表setmeal

名	类型	长度	小数点	不是 null	键	注释
id	bigint	20		☒	 1	主键
category_id	bigint	20		☒		菜品分类id
name	varchar	64		☒		套餐名称
price	decimal	10	2	☒		套餐价格
status	int	11		☐		状态 0:停用 1:启用
code	varchar	32		☐		编码
description	varchar	512		☐		描述信息
image	varchar	255		☐		图片
create_time	datetime			☒		创建时间
update_time	datetime			☒		更新时间
create_user	bigint	20		☒		创建人
update_user	bigint	20		☒		修改人
is_deleted	int	11		☒		是否删除

在该表中，套餐名称name字段是不允许重复的，在建表时，已经创建了唯一索引。

字段	索引	外键	触发器	选项	注释	SQL 预览
名		字段				
idx_setmeal_name		'name'				索引类型: UNIQUE 索引方法: BTREE

2) . 套餐菜品关系表setmeal_dish

名	类型	长度	小数点	不是 null	键	注释
id	bigint	20		☒	 1	主键
setmeal_id	varchar	32		☒		套餐id
dish_id	varchar	32		☒		菜品id
name	varchar	32		☐		菜品名称（冗余字段）
price	decimal	10	2	☐		菜品原价（冗余字段）
copies	int	11		☒		份数
sort	int	11		☒		排序
create_time	datetime			☒		创建时间
update_time	datetime			☒		更新时间
create_user	bigint	20		☒		创建人
update_user	bigint	20		☒		修改人
is_deleted	int	11		☒		是否删除

关联套餐表的主键

在该表中, 菜品的名称name, 菜品的原价price 实际上都是冗余字段, 因为我们在这张表中存储了菜品的ID(dish_id), 根据该ID我们就可以查询出name, price的数据信息, 而这里我们又存储了name, price, 这样的话, 我们在后续的查询展示操作中, 就不需要再去查询数据库获取菜品名称和原价了, 这样可以简化我们的操作。

1.3 准备工作

在开发业务功能前, 先将需要用到的类和接口基本结构创建好, 在做这一块儿的准备工作时, 我们无需准备Setmeal的相关实体类、Mapper接口、Service接口及实现, 因为之前在做分类管理的时候, 我们已经引入了Setmeal的相关基础代码。 接下来, 我们就来完成以下的几步准备工作:

1). 实体类 SetmealDish

ps. 直接从课程资料中导入即可, Setmeal实体前面课程中已经导入过了。

所属包: com.itheima.reggie.entity

```
1  import com.baomidou.mybatisplus.annotation.FieldFill;
2  import com.baomidou.mybatisplus.annotation.TableField;
3  import lombok.Data;
4  import java.io.Serializable;
5  import java.math.BigDecimal;
6  import java.time.LocalDateTime;
7
8  /**
9   * 套餐菜品关系
10  */
11  @Data
12  public class SetmealDish implements Serializable {
13      private static final long serialVersionUID = 1L;
14
15      private Long id;
16
17      //套餐id
18      private Long setmealId;
19
20      //菜品id
```

```

21     private Long dishId;
22
23     //菜品名称 （冗余字段）
24     private String name;
25
26     //菜品原价
27     private BigDecimal price;
28
29     //份数
30     private Integer copies;
31
32     //排序
33     private Integer sort;
34
35     @TableField(fill = FieldFill.INSERT)
36     private LocalDateTime createTime;
37
38     @TableField(fill = FieldFill.INSERT_UPDATE)
39     private LocalDateTime updateTime;
40
41     @TableField(fill = FieldFill.INSERT)
42     private Long createUser;
43
44     @TableField(fill = FieldFill.INSERT_UPDATE)
45     private Long updateUser;
46
47     //是否删除
48     private Integer isDeleted;
49 }

```

2). DTO SetmealDto

该数据传输对象DTO,主要用于封装页面在新增套餐时传递过来的json格式的数据,其中包含套餐的基本信息,还包含套餐关联的菜品集合。直接从课程资料中导入即可。

所属包: com.itheima.reggie.dto

```

1  import com.itheima.reggie.entity.Setmeal;
2  import com.itheima.reggie.entity.SetmealDish;
3  import lombok.Data;
4  import java.util.List;
5
6  @Data
7  public class SetmealDto extends Setmeal {
8
9      private List<SetmealDish> setmealDishes;//套餐关联的菜品集合
10
11     private String categoryName;//分类名称
12 }

```

3). Mapper接口 SetmealDishMapper

所属包: com.itheima.reggie.mapper

```

1  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
2  import com.itheima.reggie.entity.SetmealDish;
3  import org.apache.ibatis.annotations.Mapper;
4
5  @Mapper
6  public interface SetmealDishMapper extends BaseMapper<SetmealDish> {
7  }

```

4). 业务层接口 SetmealDishService

所属包: com.itheima.reggie.service

```

1  import com.baomidou.mybatisplus.extension.service.IService;
2  import com.itheima.reggie.entity.SetmealDish;
3
4  public interface SetmealDishService extends IService<SetmealDish> {
5  }

```

5). 业务层实现类 SetmealDishServiceImpl

所属包: com.itheima.reggie.service.impl

```
1  import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
2  import com.itheima.reggie.entity.SetmealDish;
3  import com.itheima.reggie.mapper.SetmealDishMapper;
4  import com.itheima.reggie.service.SetmealDishService;
5  import lombok.extern.slf4j.Slf4j;
6  import org.springframework.stereotype.Service;
7
8  @Service
9  @Slf4j
10 public class SetmealDishServiceImpl extends
    ServiceImpl<SetmealDishMapper,SetmealDish> implements SetmealDishService {
11 }
```

6). 控制层 SetmealController

套餐管理的相关业务，我们都统一在 SetmealController 中进行统一处理操作。

所属包: com.itheima.reggie.service.impl

```
1  import com.itheima.reggie.service.SetmealDishService;
2  import com.itheima.reggie.service.SetmealService;
3  import lombok.extern.slf4j.Slf4j;
4  import org.springframework.beans.factory.annotation.Autowired;
5  import org.springframework.web.bind.annotation.*;
6
7  /**
8   * 套餐管理
9   */
10 @RestController
11 @RequestMapping("/setmeal")
12 @Slf4j
13 public class SetmealController {
14     @Autowired
15     private SetmealService setmealService;
16     @Autowired
17     private SetmealDishService setmealDishService;
```

1.4 前端页面分析

服务端的基础准备工作我们准备完毕之后，在进行代码开发之前，需要梳理一下新增套餐时前端页面和服务端的交互过程：

1) . 点击新建套餐按钮，访问页面 (backend/page/combo/add.html)，页面加载发送ajax请求，请求服务端获取套餐分类数据并展示到下拉框中（已实现）

```

<el-button type="primary" @click="addSetMeal('add')">+ 新建套餐</el-button>

// 添加
addSetMeal (st) {
  if (st === 'add') {
    window.parent.menuHandle({
      id: '5',
      url: '/backend/page/combo/add.html',
      name: '添加套餐'
    }, true)
  } else {
    window.parent.menuHandle({
      id: '5',
      url: '/backend/page/combo/add.html?id='+st,
      name: '修改套餐'
    }, true)
  }
}

// 获取套餐分类
getDishTypeList() {
  getCategoryList({ type: 2 }).then((res) => {
    if (res.code === 1) {
      this.setMealList = res.data.map((obj) => ({ ...obj, idType: obj.id }))
    } else {
      this.$message.error(res.msg || '操作失败')
    }
  })
}

const getCategoryList = (params) => {
  return $axios({
    url: '/category/list',
    method: 'get',
    params
  })
}
  
```

add.html

```

created() {
  this.getDishTypeList()
  this.getDishType()
  this.id = requestUriParam('id')
  this.actionType = this.id ? 'edit' : 'add'
  if (this.id) {
    this.init()
  }
}

// 获取套餐分类
getDishTypeList() {
  getCategoryList({ type: 2 }).then((res) => {
    if (res.code === 1) {
      this.setMealList = res.data.map((obj) => ({ ...obj, idType: obj.id }))
    } else {
      this.$message.error(res.msg || '操作失败')
    }
  })
}

const getCategoryList = (params) => {
  return $axios({
    url: '/category/list',
    method: 'get',
    params
  })
}
  
```

页面下拉列表渲染展示

套餐分类

超微午餐

精品肉粽

精品肉粽

精品肉粽

精品肉粽

精品肉粽

精品肉粽

精品肉粽

获取套餐分类列表的功能我们不用开发，之前已经开发完成了，之前查询时type传递的是1，查询菜品分类；本次查询时，传递的type为2，查询套餐分类列表。

2) . 访问页面 (backend/page/combo/add.html)，页面加载时发送ajax请求，请求服务端获取菜品分类数据并展示到添加菜品窗口中（已实现）

```

created() {
  this.getDishTypeList()
  this.getDishType()
  this.id = requestUriParam('id')
  this.actionType = this.id ? 'edit' : 'add'
  if (this.id) {
    this.init()
  }
}

// 获取菜品分类
getDishType () {
  getCategoryList({ type: 1 }).then((res) => {
    if (res.code === 1) {
      this.dishType = res.data
      this.getDishList(res.data[0].id)
    } else {
      this.$message.error(res.msg)
    }
  })
}

const getCategoryList = (params) => {
  return $axios({
    url: '/category/list',
    method: 'get',
    params
  })
}
  
```

const getCategoryList = (params) => {

return \$axios({

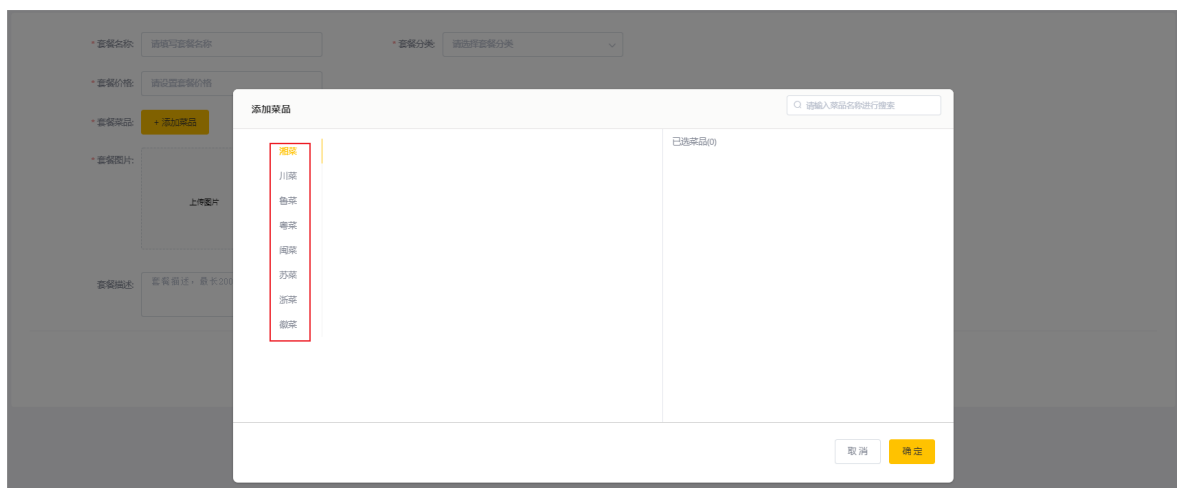
url: '/category/list',

method: 'get',

params

)

本次查询分类列表，传递的type为1，表示需要查询的是菜品的分类。查询菜品分类的目的，是添加套餐关联的菜品时，我们需要根据菜品分类，来过滤查询菜品信息。查询菜品分类列表的代码已经实现，具体展示效果如下：



3). 当点击添加菜品窗口左侧菜单的某一个分类, 页面发送ajax请求, 请求服务端, 根据菜品分类查询对应的菜品数据并展示到添加菜品窗口中

```

<span
  v-for="(item, index) in dishType"
  :key="index"
  :class="{act:index == keyInd}"
  @click="checkTypeHandle(index, item.id)">{{ item.name }}</span>

checkTypeHandle (ind,id) {
  this.keyInd = ind
  this.getDishList(id)
},

// 查菜品列表的接口
const queryDishList = (params) => {
  return $axios({
    url: '/dish/list',
    method: 'get',
    params
  })
}

// 通过套餐ID获取菜品列表分类
getDishList (id) {
  queryDishList({categoryId: id}).then(res => {
    if (res.code === 1) {
      if (res.data.length === 0) {
        this.dishAddList = []
        return
      }
      let newArr = res.data;
      newArr.forEach((n) => {
        n.dishId = n.id
        n.copies = 1
        // n.dishCopies = 1
        n.dishName = n.name
      })
      this.dishAddList = newArr
    } else {
      this.$message.error(res.msg)
    }
  })
},
  
```

点击左侧的菜品分类菜单

根据菜品分类ID, 查询菜品列表, 在页面展示

4). 页面发送请求进行**图片上传**, 请求服务端将图片保存到服务器 (**已实现**)

5). 页面发送请求进行**图片下载**, 将上传的图片进行回显 (**已实现**)

```

<el-upload
  class="avatar-uploader"
  action="/common/upload"
  :show-file-list="false"
  :on-success="handleAvatarSuccess"
  :on-change="onChange"
  ref="upload">
  </img>
  <i v-else class="el-icon-plus avatar-uploader-icon"></i>
</el-upload>

handleAvatarSuccess (response, file, fileList) {
  // this.imageUrl = response.data
  if(response.code === 0 && response.msg === '未登录'){
    window.top.location.href = '/backend/page/login/login.html'
  }else{
    this.imageUrl = `/common/download?name=${response.data}`
    this.ruleForm.image = response.data
  }
},
  
```

上传成功后调用

下载并展示图片

6). 点击保存按钮, 发送ajax请求, 将套餐相关数据以json形式提交到服务端



经过上述的页面解析及流程分析，我们发送这里需要发送的请求有5个，分别是：

- A. 根据传递的参数, 查询套餐分类列表
- B. 根据传递的参数, 查询菜品分类列表
- C. 图片上传
- D. 图片下载展示
- E. 根据菜品分类ID, 查询菜品列表
- F. 保存套餐信息

而对于以上的前4个功能我们都已经实现，所以我们接下来需要开发的功能主要是最后两项，具体的请求信息如下：

1). 根据分类ID查询菜品列表

请求	说明
请求方式	GET
请求路径	/dish/list
请求参数	?categoryId=1397844263642378242

2). 保存套餐信息

请求	说明
请求方式	POST
请求路径	/setmeal
请求参数	json格式数据

传递的json格式数据如下：

```
1  {
2      "name": "营养超值工作餐",
3      "categoryId": "1399923597874081794",
4      "price": 3800,
5      "code": "",
6      "image": "9cd7a80a-da54-4f46-bf33-af3576514cec.jpg",
7      "description": "营养超值工作餐",
8      "dishList": [],
9      "status": 1,
10     "idType": "1399923597874081794",
11     "setmealDishes": [
12         {"copies": 2, "dishId": "1423329009705463809", "name": "米饭", "price": 200},
13         {"copies": 1, "dishId": "1423328152549109762", "name": "可乐", "price": 500},
14         {"copies": 1, "dishId": "1397853890262118402", "name": "鱼香肉丝", "price": 3800}
15     ]
16 }
```

1.5 代码开发

上面我们已经分析了接下来我们需要实现的两个功能，接下来我们就需要根据上述的分析，来完成具体的功能实现。

1.5.1 根据分类查询菜品

1.5.1.1 功能实现

在当前的需求中，我们只需要根据页面传递的菜品分类的ID(categoryId)来查询菜品列表即可，我们可以直接定义一个DishController的方法，声明一个Long类型的categoryId，这样做是没问题的。但是考虑到该方法的拓展性，我们在这里定义方法时，通过Dish这个实体来接收参数。

在DishController中定义方法list，接收Dish类型的参数：

在查询时，需要根据菜品分类categoryId进行查询，并且还要限定菜品的状态为起售状态(status为1)，然后对查询的结果进行排序。

```
1  /**
2   * 根据条件查询对应的菜品数据
3   * @param dish
4   * @return
5   */
6   @GetMapping("/list")
7   public R<List<Dish>> list(Dish dish){
8       //构造查询条件
9       LambdaQueryWrapper<Dish> queryWrapper = new LambdaQueryWrapper<>();
10      queryWrapper.eq(dish.getCategoryId() != null
11          ,Dish::getCategoryId,dish.getCategoryId());
12      //添加条件，查询状态为1（起售状态）的菜品
13      queryWrapper.eq(Dish::getStatus,1);
14      //添加排序条件
15      queryWrapper.orderByAsc(Dish::getSort).orderByDesc(Dish::getUpdateTime);
16
17      List<Dish> list = dishService.list(queryWrapper);
18
19      return R.success(list);
20  }
```

1.5.1.2 功能测试

代码编写完毕，我们重新启动服务器，进行测试，可以通过debug断点跟踪的形式查看页面传递的参数封装情况，及响应给页面的数据信息。

添加菜品

请输入菜品名称进行搜索

湘菜

川菜

鲁菜

粤菜

闽菜

苏菜

浙菜

徽菜

主食

饮料

☐ 口味蛇 在售 168
☐ 翡翠辣血丸子 在售 138
☐ 全家福 在售 118
☐ 霸王别姬 在售 128
☐ 东安子鸡 在售 88
☐ 担担鱼翅 在售 48
☐ 毛氏红烧肉 在售 68
☐ 永州血鸭 在售 88
☐ 辣子鸡 在售 78

已选菜品(0)

取消

确定

添加菜品

请输入菜品名称进行搜索

湘菜

川菜

鲁菜

粤菜

闽菜

苏菜

浙菜

徽菜

主食

饮料

☐ 北冰洋 在售 5
☐ 雪碧 在售 5
☐ 可乐 在售 5

已选菜品(0)

取消

确定

1.5.2 保存套餐

1.5.2.1 功能实现

在进行套餐信息保存时，前端提交的数据，不仅包含套餐的基本信息，还包含套餐关联的菜品列表数据 `setmealDishes`。所以这个时候我们使用 `Setmeal` 就不能完成参数的封装了，我们需要在 `Setmeal` 的基本属性的基础上，再扩充一个属性 `setmealDishes` 来接收页面传递的套餐关联的菜品列表，而我们在准备工作中，导入进来的 `SetmealDto` 能够满足这个需求。

1). `SetmealController` 中定义方法 `save`，新增套餐

在该 `Controller` 的方法中，我们不仅需要保存套餐的基本信息，还需要保存套餐关联的菜品数据，所以我们需要再该方法中调用业务层方法，完成两块数据的保存。

页面传递的数据是 `json` 格式，需要在方法形参前面加上 `@RequestBody` 注解，完成参数封装。

```

1  @PostMapping
2  public R<String> save(@RequestBody SetmealDto setmealDto){
3
4      log.info("套餐信息: {}", setmealDto);
5
6      setmealService.saveWithDish(setmealDto);
7
8      return R.success("新增套餐成功");
9  }

```

2). `SetmealService` 中定义方法 `saveWithDish`

```

1  /**
2   * 新增套餐，同时需要保存套餐和菜品的关联关系
3   * @param setmealDto
4   */
5  public void saveWithDish(SetmealDto setmealDto);

```

3). SetmealServiceImpl实现方法saveWithDish

具体逻辑：

- A. 保存套餐基本信息
- B. 获取套餐关联的菜品集合，并为集合中的每一个元素赋值套餐ID(setmealId)
- C. 批量保存套餐关联的菜品集合

代码实现：

```

1  /**
2   * 新增套餐，同时需要保存套餐和菜品的关联关系
3   * @param setmealDto
4   */
5  @Transactional
6  public void saveWithDish(SetmealDto setmealDto) {
7      //保存套餐的基本信息，操作setmeal，执行insert操作
8      this.save(setmealDto);
9
10     List<SetmealDish> setmealDishes = setmealDto.getSetmealDishes();
11     setmealDishes.stream().map((item) -> {
12         item.setSetmealId(setmealDto.getId());
13         return item;
14     }).collect(Collectors.toList());
15
16     //保存套餐和菜品的关联信息，操作setmeal_dish,执行insert操作
17     setmealDishService.saveBatch(setmealDishes);
18 }

```

1.5.2.2 功能测试

录入表单数据：

debug跟踪数据封装:



setmeal

id	category_id	name	price	status	code	description
1423329486060957698	1399923597874081794	营养超值工作餐	3800.00	1		营养超值工作餐
1423338765002256385	1402146343886565378	情侣超值套餐	18800.00	1		情侣超值套餐

setmeal_dish

id	setmeal_id	dish_id	name	price	copies	sort
142332948608612352	1423329486060957698	142332900970	米饭	200.00	2	0
142332948609870643	1423329486060957698	142332815254	可乐	500.00	1	0
1423329486098706434	1423329486060957698	139785389026	鱼香肉丝	3800.00	1	0
1423338765010644993	1423338765002256385	139786189843	八宝冬瓜盅	22800.00	1	0
1423338765014839298	1423338765002256385	139786113573	酿豆腐	5800.00	1	0
1423338765019033602	1423338765002256385	142332815254	可乐	500.00	1	0
1423338765023227906	1423338765002256385	142332823593	雪碧	500.00	1	0
1423338765027422210	1423338765002256385	142332900970	米饭	200.00	2	0

2. 套餐分页查询

2.1 需求分析

系统中的套餐数据很多的时候，如果在一个页面中全部展示出来会显得比较乱，不便于查看，所以一般的系统中都会以分页的方式来展示列表数据。

查询条件 批量删除 批量启用 批量停用 新建套餐

☐ 套餐名称	图片	套餐分类	套餐价	售卖状态	最后操作时间	操作
☐ 情侣超值套餐		超值晚餐	¥ 188	启售	2021-08-06 01:43:00	修改 停售 删除
☐ 营养超值工作餐		超值午餐	¥ 38	启售	2021-08-06 01:06:08	修改 停售 删除

共 2 条 10条/页 < 1 > 前往 1 页

在进行套餐数据的分页查询时，除了传递分页参数以外，还可以传递一个可选的条件（套餐名称）。查询返回的字段中，包含套餐的基本信息之外，还有一个套餐的分类名称，在查询时，需要关联查询这个字段。

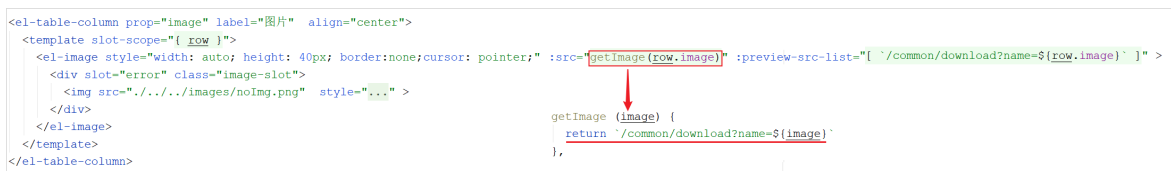
2.2 前端页面分析

在开发代码之前，需要梳理一下套餐分页查询时前端页面和服务端的交互过程：

- 1). 访问页面 (backend/page/combo/list.html)，页面加载时，会自动发送ajax请求，将分页查询参数 (page、pageSize、name) 提交到服务端，获取分页数据



2). 在列表渲染展示时, 页面发送请求, 请求服务端进行图片下载, 用于页面图片展示 (已实现)



而对于以上的流程中涉及到2个功能, 文件下载功能我们已经实现, 本小节我们主要实现列表分页查询功能, 具体的请求信息如下:

请求	说明
请求方式	GET
请求路径	/setmeal/page
请求参数	?page=1&pageSize=10&name=xxx

2.3 代码开发

2.3.1 基本信息查询

上述我们已经分析列表分页查询功能的请求信息, 接下来我们就在SetmealController中创建套餐分页查询方法。

该方法的逻辑如下:

1). 构建分页条件对象

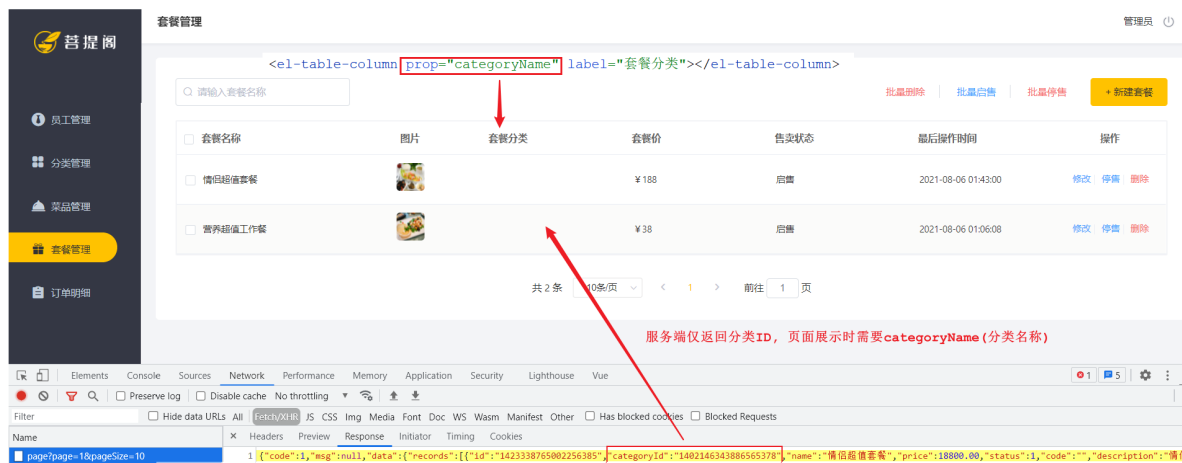
- 2). 构建查询条件对象，如果传递了套餐名称，根据套餐名称模糊查询， 并对结果按修改时间降序排序
- 3). 执行分页查询
- 4). 组装数据并返回

代码实现：

```
1  /**
2   * 套餐分页查询
3   * @param page
4   * @param pageSize
5   * @param name
6   * @return
7   */
8  @GetMapping("/page")
9  public R<Page> page(int page,int pageSize,String name){
10     //分页构造器对象
11     Page<Setmeal> pageInfo = new Page<>(page,pageSize);
12
13     LambdaQueryWrapper<Setmeal> queryWrapper = new LambdaQueryWrapper<>();
14     //添加查询条件，根据name进行like模糊查询
15     queryWrapper.like(name != null,Setmeal::getName,name);
16     //添加排序条件，根据更新时间降序排列
17     queryWrapper.orderByDesc(Setmeal::getUpdateTime);
18
19     setmealService.page(pageInfo,queryWrapper);
20     return R.success(pageInfo);
21 }
```

2.3.2 问题分析

基本分页查询代码编写完毕后，重启服务，测试列表查询，我们发现，列表页面的数据可以展示出来，但是套餐分类名称没有展示出来。



这是因为在服务端仅返回分类ID (categoryId)，而页面展示需要的是categoryName属性。

2.3.3 功能完善

在查询套餐信息时，只包含套餐的基本信息，并不包含套餐的分类名称，所以在这里查询到套餐的基本信息后，还需要根据分类ID (categoryId)，查询套餐分类名称 (categoryName)，并最终将套餐的基本信息及分类名称信息封装到SetmealDto (在第一小节已经导入) 中。

```
1  @Data
2  public class SetmealDto extends Setmeal {
3      private List<SetmealDish> setmealDishes; //套餐关联菜品列表
4      private String categoryName; //套餐分类名称
5  }
```

完善后代码：

```
1  /**
2   * 套餐分页查询
3   * @param page
4   * @param pageSize
5   * @param name
6   * @return
7   */
8  @GetMapping("/page")
9  public R<Page> page(int page,int pageSize,String name){
10     //分页构造器对象
11     Page<Setmeal> pageInfo = new Page<>(page,pageSize);
12     Page<SetmealDto> dtoPage = new Page<>();
```

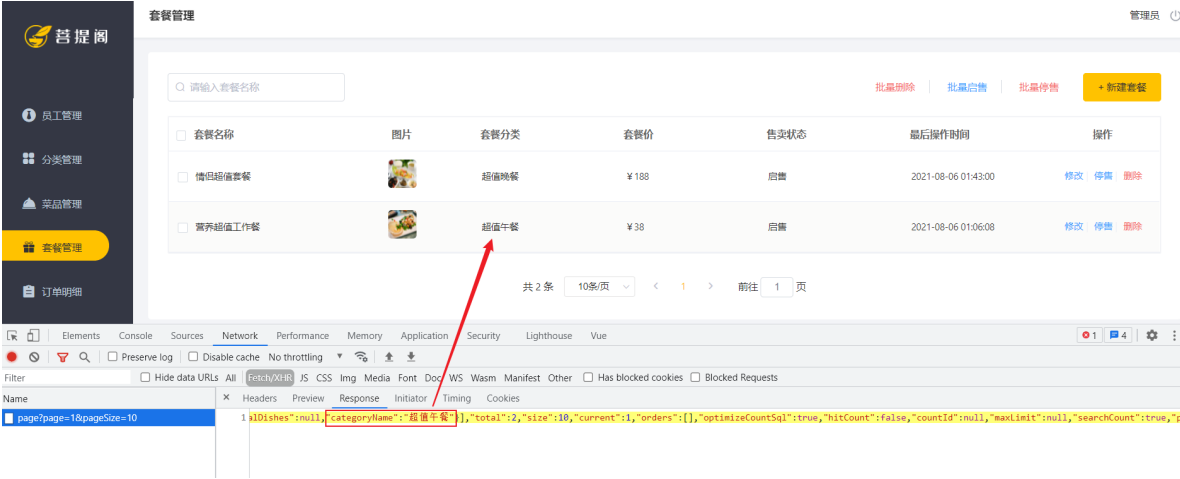
```

13
14     LambdaQueryWrapper<Setmeal> queryWrapper = new LambdaQueryWrapper<>();
15     //添加查询条件, 根据name进行like模糊查询
16     queryWrapper.like(name != null, Setmeal::getName, name);
17     //添加排序条件, 根据更新时间降序排列
18     queryWrapper.orderByDesc(Setmeal::getUpdateTime);
19
20     setmealService.page(pageInfo, queryWrapper);
21
22     //对象拷贝
23     BeanUtils.copyProperties(pageInfo, dtoPage, "records");
24     List<Setmeal> records = pageInfo.getRecords();
25
26     List<SetmealDto> list = records.stream().map((item) -> {
27         SetmealDto setmealDto = new SetmealDto();
28         //对象拷贝
29         BeanUtils.copyProperties(item, setmealDto);
30         //分类id
31         Long categoryId = item.getCategoryId();
32         //根据分类id查询分类对象
33         Category category = categoryService.getById(categoryId);
34         if(category != null){
35             //分类名称
36             String categoryName = category.getName();
37             setmealDto.setCategoryName(categoryName);
38         }
39         return setmealDto;
40     }).collect(Collectors.toList());
41
42     dtoPage.setRecords(list);
43     return R.success(dtoPage);
44 }

```

2.4 功能测试

代码完善后，重启服务，测试列表查询，我们发现，抓取浏览器的请求响应数据，我们可以获取到套餐分类名称categoryName，也可以在列表页面展示出来。



3. 删除套餐

3.1 需求分析

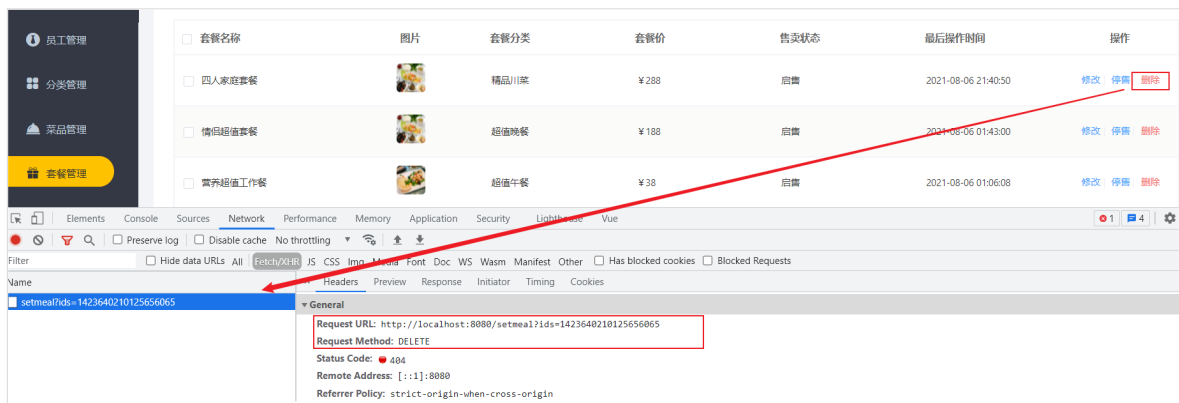
在套餐管理列表页面，点击删除按钮，可以删除对应的套餐信息。也可以通过复选框选择多个套餐，点击批量删除按钮一次删除多个套餐。注意，对于状态为售卖中的套餐不能删除，需要先停售，然后才能删除。



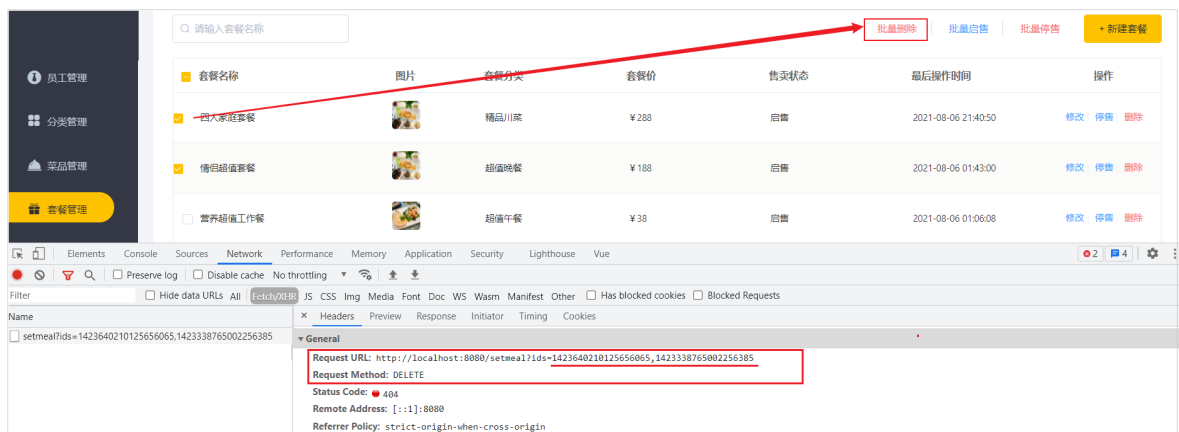
3.2 前端页面分析

在开发代码之前，需要梳理一下删除套餐时前端页面和服务端的交互过程：

- 1). 点击删除，删除单个套餐时，页面发送ajax请求，根据套餐id删除对应套餐



2). 删除多个套餐时，页面发送ajax请求，根据提交的多个套餐id删除对应套餐



开发删除套餐功能，其实就是在服务端编写代码去处理前端页面发送的这2次请求即可，一次请求为根据ID删除，一次请求为根据ID批量删除。

观察删除单个套餐和批量删除套餐的请求信息可以发现，两种请求的地址和请求方式都是相同的，不同的则是传递的id个数，所以在服务端可以提供一个方法来统一处理。

具体的请求信息如下：

请求	说明
请求方式	DELETE
请求路径	/setmeal
请求参数	?ids=1423640210125656065,1423338765002256385

3.3 代码开发

删除套餐的流程及请求信息，我们分析完毕之后，就来完成服务端的逻辑开发。在服务端的逻辑中，删除套餐时，我们不仅要删除套餐，还要删除套餐与菜品的关联关系。

1). 在SetmealController中创建delete方法

我们可以先测试在delete方法中接收页面提交的参数，具体逻辑后续再完善：

```
1  /**
2   * 删除套餐
3   * @param ids
4   * @return
5   */
6  @DeleteMapping
7  public R<String> delete(@RequestParam List<Long> ids){
8      log.info("ids:{",ids);
9      return R.success("套餐数据删除成功");
10 }
```

编写完代码，我们重启服务之后，访问套餐列表页面，勾选复选框，然后点击"批量删除"，我们可以看到服务端可以接收到集合参数ids，并且在控制台也可以输出对应的数据。

```
INFO 9916 --- [nio-8080-exec-7] c.i.reggie.filter.LoginCheckFilter : 拦截到请求: /setmeal
INFO 9916 --- [nio-8080-exec-7] c.i.reggie.filter.LoginCheckFilter : 用户已登录, 用户id为: 1412578435737350122
INFO 9916 --- [nio-8080-exec-7] c.i.reggie.controller.SetmealController : ids:[1423640210125656065, 1423338765002256385]
INFO 9916 --- [nio-8080-exec-8] c.i.reggie.filter.LoginCheckFilter : 拦截到请求: /setmeal/page
INFO 9916 --- [nio-8080-exec-8] c.i.reggie.filter.LoginCheckFilter : 用户已登录, 用户id为: 1412578435737350122
```

2). SetmealService接口定义方法removeWithDish

```
1  /**
2   * 删除套餐，同时需要删除套餐和菜品的关联数据
3   * @param ids
4   */
5  public void removeWithDish(List<Long> ids);
```

3). SetmealServiceImpl中实现方法removeWithDish

该业务层方法具体的逻辑为：

- A. 查询该批次套餐中是否存在售卖中的套餐，如果存在，不允许删除
- B. 删除套餐数据

C. 删除套餐关联的菜品数据

代码实现为：

```
1  /**
2   * 删除套餐，同时需要删除套餐和菜品的关联数据
3   * @param ids
4   */
5   @Transactional
6   public void removeWithDish(List<Long> ids) {
7       //select count(*) from setmeal where id in (1,2,3) and status = 1
8       //查询套餐状态，确定是否可用删除
9       LambdaQueryWrapper<Setmeal> queryWrapper = new LambdaQueryWrapper();
10      queryWrapper.in(Setmeal::getId,ids);
11      queryWrapper.eq(Setmeal::getStatus,1);
12
13      int count = this.count(queryWrapper);
14      if(count > 0){
15          //如果不能删除，抛出一个业务异常
16          throw new CustomException("套餐正在售卖中，不能删除");
17      }
18
19      //如果可以删除，先删除套餐表中的数据---setmeal
20      this.removeByIds(ids);
21
22      //delete from setmeal_dish where setmeal_id in (1,2,3)
23      LambdaQueryWrapper<SetmealDish> lambdaQueryWrapper = new LambdaQueryWrapper<>
24      ();
25      lambdaQueryWrapper.in(SetmealDish::getSetmealId,ids);
26      //删除关系表中的数据----setmeal_dish
27      setmealDishService.remove(lambdaQueryWrapper);
28  }
```

由于当前的业务方法中存在多次数据库操作，为了保证事务的完整性，需要在方法上加注解 `@Transactional` 来控制事务。

4). 完善SetmealController代码

```
1  /**
2   * 删除套餐
3   * @param ids
4   * @return
5   */
6  @DeleteMapping
7  public R<String> delete(@RequestParam List<Long> ids) {
8      log.info("ids:{",ids);
9      setmealService.removeWithDish(ids);
10     return R.success("套餐数据删除成功");
11 }
```

3.4 功能测试

代码完善后，重启服务，测试套餐的删除功能，主要测试以下几种情况。

1). 删除正在启用的套餐



2). 执行批量操作，删除两条记录，一个启售的，一个停售的

由于当前我们并未实现启售/停售功能，所以我们需要手动修改数据库表结构的status状态，将其中的一条记录status修改为0。



3). 删除已经停售的套餐信息, 执行删除之后, 检查数据库表结构 `setmeal` , `setmeal_dish` 表中的数据

套餐名称	图片	套餐分类	套餐价	售卖状态	最后操作时间	操作
情侣超值套餐		超值晚餐	¥188	售罄	2021-08-06 01:43:00	修改 停售 删除
营养超值工作餐		超值午餐	¥38	售罄	2021-08-06 01:06:08	修改 停售 删除

4. 短信发送

登录

请输入手机号码 获取验证码

请输入验证码

登录

菩提阁餐厅

距离1.5km 配送费6元 预计时长12min

简介: 菩提阁中餐厅是菩提阁点餐的独立的品牌, 定位“大众”化的美食外送餐饮, 旨在为顾客打造专业美食。

精品热菜

龙井虾仁

龙井虾仁将茶饮与虾... 月销0

¥99.8 选择规格

湘菜

精品闽菜

小米炖辽参

参富含蛋白质, 具有... 月销0

¥488 +

超值晚餐

老鸭汤

秋日里, 来一碗, 滋... 月销0

¥86.5 +

精品湘菜

精品川菜

秋日螃蟹

来自老香港神秘味道 月销0

¥566 +

川菜

¥148 去结算

在我们接下来要实现的移动端的业务开发中，第一块儿我们需要开发的功能就是移动端的登录功能，而移动端的登录功能，比较流行的方式就是基于短信验证码进行登录，那么这里涉及到了短信发送的知识，所以本章节，我们就来讲解，在项目开发中，我们如何发送短信。

4.1 短信服务介绍

在项目中，如果我们要实现短信发送功能，我们无需自己实现，也无需和运营商直接对接，只需要调用第三方提供的短信服务即可。目前市面上有很多第三方提供的短信服务，这些第三方短信服务会和各个运营商（移动、联通、电信）对接，我们只需要注册成为会员，并且按照提供的开发文档进行调用就可以发送短信。需要说明的是，这些短信服务一般都是收费服务。

常用短信服务：

- 阿里云
- 华为云
- 腾讯云
- 京东
- 梦网
- 乐信

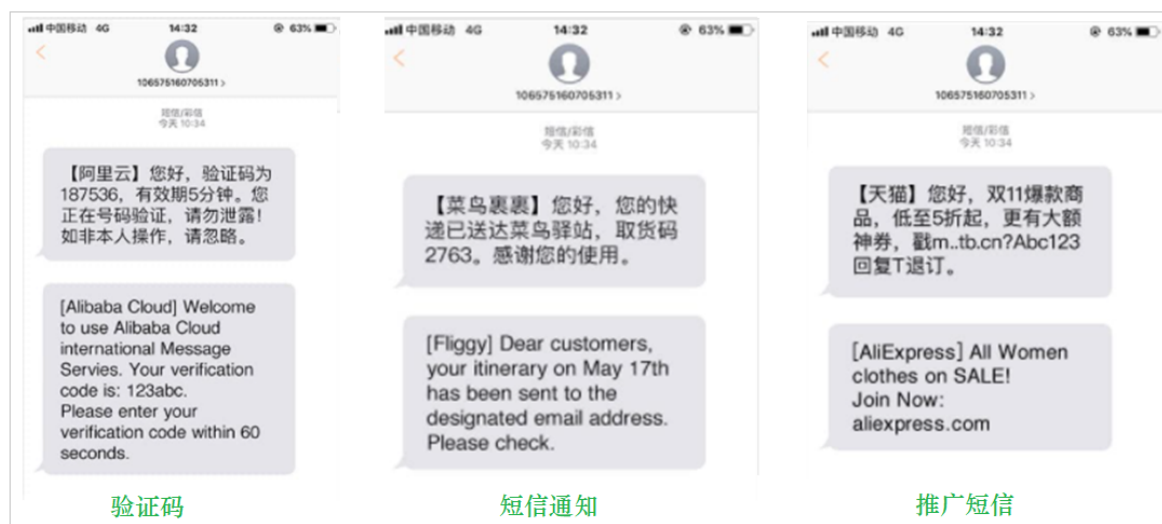
本项目在选择短信服务的第三方服务提供商时，选择的是阿里云短信服务。

4.2 阿里云短信服务介绍

阿里云短信服务（Short Message Service）是广大企业客户快速触达手机用户所优选使用的通信能力。调用API或用群发助手，即可发送验证码、通知类和营销类短信；国内验证短信秒级触达，到达率最高可达99%；国际/港澳台短信覆盖200多个国家和地区，安全稳定，广受出海企业选用。

应用场景：

场景	案例
验证码	APP、网站注册账号，向手机下发验证码； 登录账户、异地登录时的安全提醒； 找回密码时的安全验证； 支付认证、身份校验、手机绑定等。
短信通知	向注册用户下发系统相关信息，包括： 升级或维护、服务开通、价格调整、 订单确认、物流动态、消费确认、 支付通知等普通通知短信。
推广短信	向注册用户和潜在客户发送通知和推广信息，包括促销活动通知、业务推广等商品与活动的推广信息。增加企业产品曝光率、提高产品的知名度。



阿里云短信服务官方网站：https://www.aliyun.com/product/sms?spm=5176.19720258.J_8058803260.52.5c432c4a11Dcwf

可以访问官网，熟悉一下短信服务：



4.3 阿里云短信服务准备

4.3.1 注册账号

阿里云官网：<https://www.aliyun.com/>



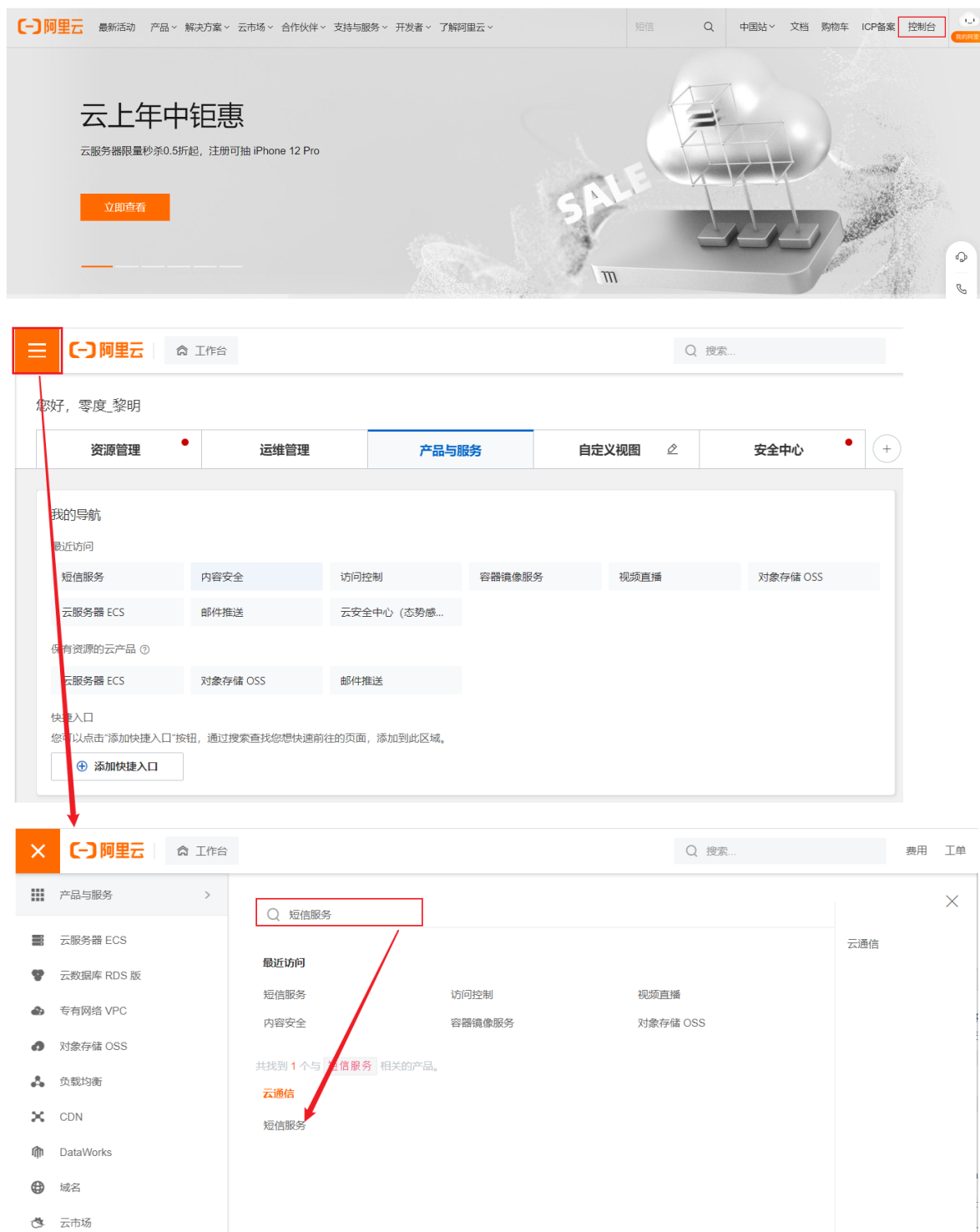
点击官网首页注册按钮，跳转到如下注册页面：



当我们把账号注册完毕之后，我们就可以登录到阿里云系统控制台。

4.3.2 开通短信服务

注册成功后，点击登录按钮进行登录。登录后进入控制台，在左上角的菜单栏中搜索短信服务。第一次使用，需要点击，并开通短信服务。



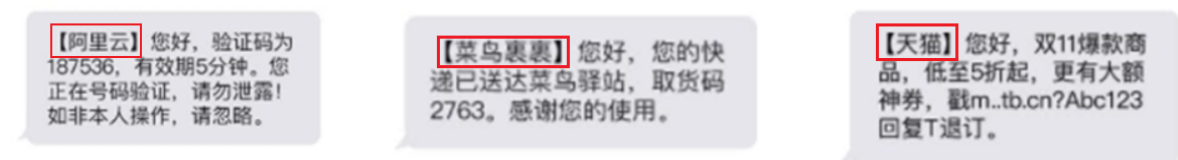
4.3.3 设置短信签名

开通短信服务之后，进入短信服务管理页面，选择国内消息菜单，我们需要在这里添加短信签名。



那么什么是短信签名呢？

短信签名是短信发送者的署名，表示发送方的身份。我们要调用阿里云短信服务发送短信，签名是比不可少的部分。



那么接下来，我们就需要来添加短信签名。

签名管理

模版管理

群发助手

请输入模版名称或模版CODE搜索

查询

标签筛选

添加模版

☐	模版名称	标签	工单号	模版CODE	模版类型(全部) ▽	创建时间	审核状态(全部) ▽	操作
☐	瑞吉外卖		141759486	SMS_210065916	验证码		通过	详情 复制 群发 删除
☐	传智健康预约验证码		136614089	SMS_204105651	验证码		通过	详情 复制 群发 删除

那么什么是模板呢？

短信模板包含短信发送内容、场景、变量信息。模板的详情如下：

短信模版详情

模版类型:

验证码

模版名称:

传智健康预约验证码

模版CODE:

SMS_204105651

标签:

模版内容:

您好, 您的验证码为 \${code} , 5分钟之内有效, 不要泄露给他人!

申请说明:

测试

确定

取消

最终我们，给用户发送的短信中，具体的短信内容，就是上面配置的这个模板内容，将`${code}` 占位符替换成对应的验证码数据即可。如下：

1 【xxxxxx】您好, 您的验证码为173822, 5分钟之内有效, 不要泄露给他人!

我们可以点击右上角的按钮，添加模板，然后填写模板的基本信息及设置的模板内容：

* 模版类型:

☒ 短信通知 (0.045元/条)

☐ 验证码 (0.045元/条)

☐ 推广短信 (0.055元/条) [升级为企业后启用](#)

为提升审核通过率, 建议您先申请签名再申请模板

* 模版名称:

请输入名称 不超过30个字符

0/30

* 模版内容:

变量格式: `${name}`;
示例: 尊敬的`${name}`, 您的快递已在飞奔的路上, 将在今天`${time}`送达您的手里, 请留意查收。

0/500

i

短信字数含“签名+模版内容+变量内容”, 短信70个字数含以内, 按1条短信计费; 超出70个字为长短信, 按照67个字数记为1条短信费用。

想快速获得可用模版, 可使用[常用模版库](#)

- 不能发送营销/贷款/借款/中奖/抽奖类短信, 不支持金融理财&房产通知类短信(验证码除外)
- [点击查看签名/模版申请规范](#)

* 申请说明:

请提供您已上线业务的对应链接 (必填)、使用场景说明
对应链接: 如官网、应用市场下载页等, 内网环境链接可提交测试账号及密码。
使用场景: 如场景、事例等详细说明

0/500

请提供您已上线业务的对应链接 (必填)、使用场景说明

暂时无法支持未上线应用、未备案域名的模板申请

提交

模版预览

- 预计两小时完成审核
- 审核工作时间: 周一至周日9:00-23:00 (法定节假日顺延)

添加的短信模板，也是需要进行审核的只有审核通过，才可以正常使用。

4.3.5 设置AccessKey

AccessKey 是访问阿里云 API 的密钥，具有账户的完全权限，我们要想在后面通过API调用阿里云短信服务的接口发送短信，那么就必须要设置AccessKey。

我们点击右上角的用户头像，选择"AccessKey管理"，这时就可以进入到AccessKey的管理界面。



进入到AccessKey的管理界面之后，提示两个选项 "继续使用AccessKey" 和 "开始使用子用户AccessKey"，两个区别如下：

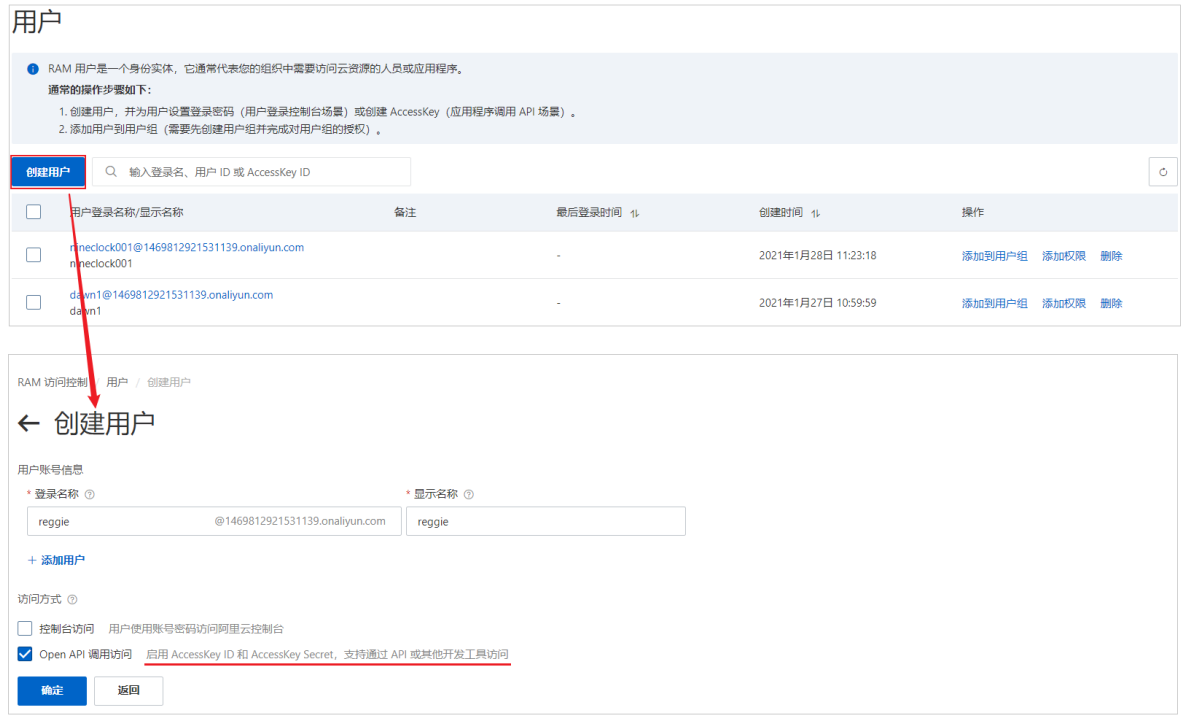
1) . 继续使用AccessKey

如果选择的是该选项，我们创建的是阿里云账号的AccessKey，是具有账户的完全权限，有了这个AccessKey以后，我们就可以通过API调用阿里云的服务，不仅是短信服务，其他服务（OSS，语音服务，内容安全服务，视频点播服务...等）也可以调用。相对来说，并不安全，当前的AccessKey泄露，会影响到我当前账户的其他云服务。

2) . 开始使用子用户AccessKey

可以创建一个子用户，这个子用户我们可以分配比较低的权限，比如仅分配短信发送的权限，不具备操作其他的服务的权限，即使这个AccessKey泄露了，也不会影响其他的云服务，相对安全。

接下来就来演示一下，如何创建子用户AccessKey。



4.3.6 配置权限

上述我们已经创建了子用户，但是这个子用户，目前没有任何权限，接下来，我们需要为创建的这个用户来分配权限。

☐	用户登录名称/显示名称	备注	最后登录时间	创建时间	操作
☐	reggie@1469812921531139.onaliyun.com reggie	选中账号, 点击	-	2021年8月7日 18:33:41	添加到用户组 添加权限 删除
☐	nineclock001@1469812921531139.onaliyun.com nineclock001		-	2021年1月28日 11:23:18	添加到用户组 添加权限 删除
☐	dawn1@1469812921531139.onaliyun.com dawn1		-	2021年1月27日 10:59:59	添加到用户组 添加权限 删除

RAM 访问控制 / 用户 / reggie@1469812921531139.onaliyun.com

← reggie@1469812921531139.onaliyun.com

用户基本信息

编辑基本信息

登录名称

reggie@1469812921531139.onaliyun.com

显示名称

reggie

备注

邮箱

认证管理

加入的组

权限管理

个人权限

继承用户组的权限

添加权限

权限应用范围

权限策略名称

权限

添加权限

指定资源组的授权生效前提是该云服务已支持资源组。查看当前支持资源组的云服务。[前往查看]

单次授权最多支持 5 条策略，如需绑定更多策略，请分多次进行。

* 授权应用范围

☒ 整个云账号

☐ 指定资源组

请选择或输入资源组名称进行搜索

* 被授权主体

reggie@1469812921531139.onaliyun.com

* 选择权限

系统策略

自定义策略

+ 新建权限策略

SMS

权限策略名称

备注

AliyunDysmsFullAccess

管理短信服务(SMS)的权限

AliyunDysmsReadOnlyAccess

只读访问短信服务(SMS)的权限

已选择 (2)

清空

AliyunDysmsFullAccess

×

AliyunDysmsReadOnlyAccess

×

确定

取消

经过上述的权限配置之后，那么新创建的这个 reggie 用户，仅有短信服务操作的权限，不具备别的权限，即使当前的AccessKey泄漏了，也只会影响短信服务，其他服务是不受影响的。

4.3.7 禁用/删除AccessKey

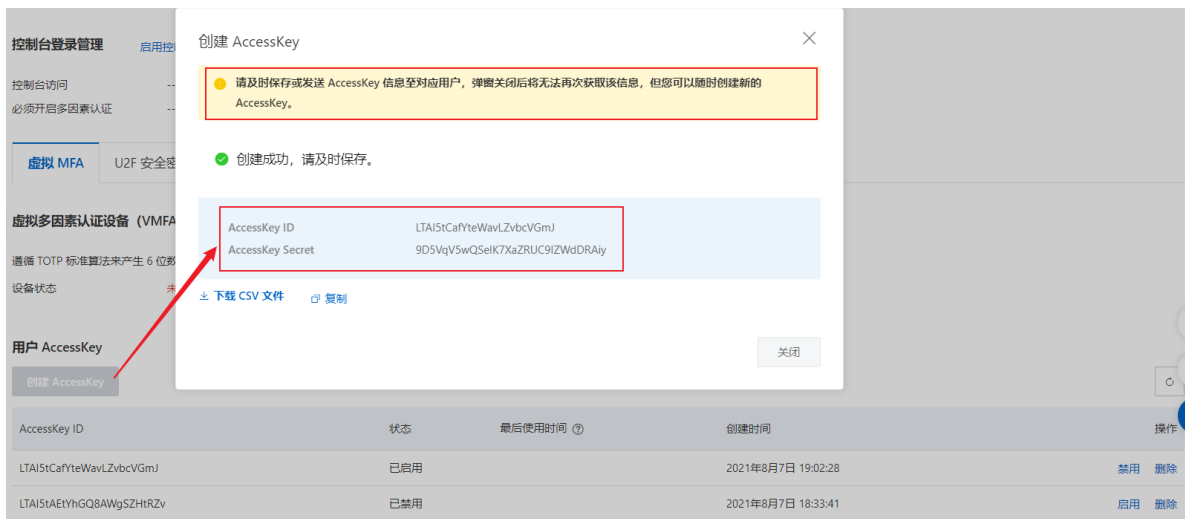
如果在使用的过程中 AccessKey 不小心泄漏了，我们可以在阿里云控制台中，禁用或者删除该 AccessKey。

用户 AccessKey

创建 AccessKey

AccessKey ID	状态	最后使用时间	创建时间	操作
LTAI5tAEtYhGQ8AWgSZHtRZv	已启用		2021年8月7日 18:33:41	禁用 删除

然后再创建一个新的AccessKey，保存好AccessKeyId和AccessKeySecret。



注意：创建好了AccessKey后，请及时保存AccessKeyId 和 AccessKeySecret ，弹窗关闭后将无法再次获取该信息，但您可以随时创建新的 AccessKey。

4.4 代码开发

使用阿里云短信服务发送短信，可以参照官方提供的文档即可。

官方文档：<https://help.aliyun.com/product/44282.html?spm=5176.12212571.help.dexternal.57a91cbewHHjKq>



我们根据官方文档的提示，引入对应的依赖，然后再引入对应的java代码，就可以发送消息了。

```
public class SendSms {
    public static void main(String[] args) {
        DefaultProfile profile = DefaultProfile.getProfile("cn-hangzhou", "<accessKeyId>", "<accessSecret>");
        IAcsClient client = new DefaultAcsClient(profile);

        CommonRequest request = new CommonRequest();
        request.setSysMethod(MethodType.POST);
        request.setSysDomain("dysmsapi.aliyuncs.com");
        request.setSysVersion("2017-05-25");
        request.setSysAction("SendSms");
        request.putQueryParam("PhoneNumbers", "1503871****");
        request.putQueryParam("SignName", "阿里大于测试专用");
        request.putQueryParam("TemplateCode", "SMS_209335004");
        request.putQueryParam("TemplateParam", "{\\code\\:\\1111\\}");
        try {
            CommonResponse response = client.getCommonResponse(request);
            System.out.println(response.getData());
        } catch (ServerException e) {
            e.printStackTrace();
        } catch (ClientException e) {
            e.printStackTrace();
        }
    }
}
```

签名名称	适用场景
瑞吉外卖	验证码
品优购商城	通用

模板名称	工单号	模板CODE	模板类型(全部)
瑞吉外卖	141759486	SMS_210065916	验证码
传智健康预约验证码	136614089	SMS_204105651	验证码

SDK：SDK 就是 Software Development Kit 的缩写，翻译过来—软件开发工具包，辅助开发某一类软件的相关文档、范例和工具的集合都可以叫做SDK。在我们与第三方接口相互时，一般都会提供对应的SDK，来简化我们的开发。

具体实现：

1) . pom.xml

```
1 <dependency>
2     <groupId>com.aliyun</groupId>
3     <artifactId>aliyun-java-sdk-core</artifactId>
4     <version>4.5.16</version>
5 </dependency>
6 <dependency>
7     <groupId>com.aliyun</groupId>
8     <artifactId>aliyun-java-sdk-dysmsapi</artifactId>
9     <version>2.1.0</version>
10 </dependency>
```

2) . 将官方提供的main方法封装为一个工具类

```
1 import com.aliyuncs.DefaultAcsClient;
2 import com.aliyuncs.IAcsClient;
3 import com.aliyuncs.dysmsapi.model.v20170525.SendSmsRequest;
4 import com.aliyuncs.dysmsapi.model.v20170525.SendSmsResponse;
5 import com.aliyuncs.exceptions.ClientException;
6 import com.aliyuncs.profile.DefaultProfile;
```

```

7
8  /**
9   * 短信发送工具类
10  */
11  public class SMSUtils {
12      /**
13       * 发送短信
14       * @param signName 签名
15       * @param templateCode 模板
16       * @param phoneNumbers 手机号
17       * @param param 参数
18       */
19      public static void sendMessage(String signName, String templateCode, String
phoneNumbers, String param) {
20          DefaultProfile profile = DefaultProfile.getProfile("cn-hangzhou",
"xxxxxxxxxxxxxxxx", "xxxxxxxxxxxxxxxx");
21          IAcsClient client = new DefaultAcsClient(profile);
22
23          SendSmsRequest request = new SendSmsRequest();
24          request.setSysRegionId("cn-hangzhou");
25          request.setPhoneNumbers(phoneNumbers);
26          request.setSignName(signName);
27          request.setTemplateCode(templateCode);
28          request.setTemplateParam("{\"code\":\""+param+"\"}");
29          try {
30              SendSmsResponse response = client.getAcsResponse(request);
31              System.out.println("短信发送成功");
32          } catch (ClientException e) {
33              e.printStackTrace();
34          }
35      }
36
37  }

```

备注：由于我们个人目前无法申请阿里云短信服务，所以这里我们只需要把流程跑通，具体的短信发送可以实现。

5. 手机验证码登录

5.1 需求分析

为了方便用户登录，移动端通常都会提供通过手机验证码登录的功能。手机验证码登录有如下优点：

- 1). 方便快捷，无需注册，直接登录
- 2). 使用短信验证码作为登录凭证，无需记忆密码
- 3). 安全

The image shows a mobile app login screen. At the top is a dark grey header with the word '登录' (Login) in white. Below the header is a light grey container. Inside this container, there is a white box with two input fields. The first field is labeled '请输入手机号码' (Please enter phone number) and has a red-bordered button labeled '获取验证码' (Get verification code) to its right. The second field is labeled '请输入验证码' (Please enter verification code). Below these fields is a large, rounded grey button labeled '登录' (Login), which is also outlined with a red border.

登录流程：

输入手机号 > 获取验证码 > 输入验证码 > 点击登录 > 登录成功

注意：通过手机验证码登录，手机号是区分不同用户的标识。

5.2 数据模型

通过手机验证码登录时，涉及的表为user表，即用户表。结构如下：

字段	索引	外键	触发器	选项	注释	SQL 预览
名						类型 长度 小数点 不是 null 键 注释
id						bigint 20 主键
name						varchar 50 姓名
phone						varchar 100 手机号
sex						varchar 2 性别
id_number						varchar 18 身份证号
avatar						varchar 500 头像
status						int 11 状态 0:禁用, 1:正常

5.3 前端页面分析

在开发代码之前，需要梳理一下登录时前端页面和服务端的交互过程：

- 1). 在登录页面(front/page/login.html)输入手机号，点击【获取验证码】按钮，页面发送ajax请求，在服务端调用短信服务API给指定手机号发送验证码短信。

```

<span @click="getCode">获取验证码</span>
getCode() {
  this.form.code = ''
  const regex = /^(13[0-9]{9})|(15[0-9]{9})|(17[0-9]{9})|(18[0-9]{9})|(19[0-9]{9})$/;
  if (regex.test(this.form.phone)) {
    this.msgFlag = false
    sendMsgApi({phone:this.form.phone})
  } else {
    this.msgFlag = true
  }
}
},
function sendMsgApi(data) {
  return $axios({
    'url': '/user/sendMsg',
    'method': 'post',
    data
  })
}

```

正则校验手机号

- 2). 在登录页面输入验证码，点击【登录】按钮，发送ajax请求，在服务端处理登录请求。

```

<el-button type="primary" :class="{btnSubmit:l===1,btnNoPhone:!form.phone,btnPhone:form.phone}"
  @click="btnLogin">登录</el-button>
async btnLogin() {
  if(this.form.phone && this.form.code){
    this.loading = true
    const res = await loginApi(this.form)
    this.loading = false
    if(res.code === 1){
      sessionStorage.setItem("userPhone",this.form.phone)
      window.requestAnimationFrame(()=>{
        window.location.href= '/front/index.html'
      })
    } else {
      this.$notify({ type:'warning', message:res.msg});
    }
  } else {
    this.$notify({ type:'warning', message:'请输入手机号码'});
  }
}
}
function loginApi(data) {
  return $axios({
    'url': '/user/login',
    'method': 'post',
    data
  })
}

```

如果服务端返回的登录成功，页面将会把当前登录用户的手机号存储在sessionStorage中，并跳转到移动的首页页面。

开发手机验证码登录功能，其实就是在服务端编写代码去处理前端页面发送的这2次请求即可，分别是获取短信验证码 和 登录请求，具体的请求信息如下：

1). 获取短信验证码

请求	说明
请求方式	POST
请求路径	/user/sendMsg
请求参数	{"phone":"13100001111"}

2). 登录

请求	说明
请求方式	POST
请求路径	/user/login
请求参数	{"phone":"13100001111", "code":"1111"}

5.4 代码开发

5.4.1 准备工作

在开发业务功能前，先将需要用到的类和接口基本结构创建好：

1). 实体类 User（直接从课程资料中导入即可）

所属包：com.itheima.reggie.entity

```
1  import lombok.Data;
2  import java.io.Serializable;
3  /**
4   * 用户信息
5   */
6  @Data
7  public class User implements Serializable {
8      private static final long serialVersionUID = 1L;
9
10     private Long id;
11 }
```

```

12      //姓名
13      private String name;
14
15      //手机号
16      private String phone;
17
18      //性别 0 女 1 男
19      private String sex;
20
21      //身份证号
22      private String idNumber;
23
24      //头像
25      private String avatar;
26
27      //状态 0:禁用, 1:正常
28      private Integer status;
29  }

```

2). Mapper接口 UserMapper

所属包: com.itheima.reggie.mapper

```

1  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
2  import com.itheima.reggie.entity.User;
3  import org.apache.ibatis.annotations.Mapper;
4
5  @Mapper
6  public interface UserMapper extends BaseMapper<User>{
7  }

```

3). 业务层接口 UserService

所属包: com.itheima.reggie.service

```
1  import com.baomidou.mybatisplus.extension.service.IService;
2  import com.itheima.reggie.entity.User;
3
4  public interface UserService extends IService<User> {
5  }
```

4). 业务层实现类 UserServiceImpl

所属包: com.itheima.reggie.service.impl

```
1  import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
2  import com.itheima.reggie.entity.User;
3  import com.itheima.reggie.mapper.UserMapper;
4  import com.itheima.reggie.service.UserService;
5  import org.springframework.stereotype.Service;
6
7  @Service
8  public class UserServiceImpl extends ServiceImpl<UserMapper,User> implements
    UserService{
9  }
```

5). 控制层 UserController

所属包: com.itheima.reggie.controller

```

1  import com.itheima.reggie.service.UserService;
2  import lombok.extern.slf4j.Slf4j;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.web.bind.annotation.RequestMapping;
5  import org.springframework.web.bind.annotation.RestController;
6
7  @RestController
8  @RequestMapping("/user")
9  @Slf4j
10 public class UserController {
11     @Autowired
12     private UserService userService;
13 }

```

6). 工具类SMSUtils、ValidateCodeUtils (直接从课程资料中导入即可)

所属包: com.itheima.reggie.utils

▼ utils

- SMSUtils
- ValidateCodeUtils

SMSUtils : 是我们上面改造的阿里云短信发送的工具类 ;

ValidateCodeUtils : 是验证码生成的工具类 ;

5.4.2 功能实现

5.4.2.1 修改LoginCheckFilter

前面我们已经完成了LoginCheckFilter过滤器的开发, 此过滤器用于检查用户的登录状态。我们在进行手机验证码登录时, 发送的两个请求(获取验证码和登录)需要在此过滤器处理时直接放行。

```

LoginCheckFilter.java
24  @Override
25  public void doFilter(ServletRequest servletRequest, ServletResponse servletResponse, FilterChain filterChain) throws IOException,
26      ServletException {
27      HttpServletRequest request = (HttpServletRequest) servletRequest;
28      HttpServletResponse response = (HttpServletResponse) servletResponse;
29
30      //1、获取本次请求的URI
31      String requestURI = request.getRequestURI(); // /backend/index.html
32
33      log.info("拦截到请求: {}", requestURI);
34
35      //定义不需要处理的请求路径
36      String[] urls = new String[]{
37          "/employee/login",
38          "/employee/logout",
39          "/backend/**",
40          "/front/**",
41          "/common/**",
42          "/user/sendMsg",
43          "/user/login"
44      };

```

放行这两个请求

对于移动的端的页面，也是用户登录之后，才可以访问的，那么这个时候就需要在 `LoginCheckFilter` 中进行判定，如果移动端用户已登录，我们获取到用户登录信息，存入 `ThreadLocal`中(在后续的业务处理中，如果需要获取当前登录用户ID，直接从`ThreadLocal`中获取)，然后放行。

增加如下逻辑：

```
1 //4-2、判断登录状态，如果已登录，则直接放行
2 if(request.getSession().getAttribute("user") != null){
3     log.info("用户已登录，用户id为: {}",request.getSession().getAttribute("user"));
4
5     Long userId = (Long) request.getSession().getAttribute("user");
6     BaseContext.setCurrentId(userId);
7
8     filterChain.doFilter(request,response);
9     return;
10 }
```

5.4.2.2 发送短信验证码

在`UserController`中创建方法，处理登录页面的请求，为指定手机号发送短信验证码，同时需要将手机号对应的验证码保存到`Session`，方便后续登录时进行比对。

```
1 /**
2  * 发送手机短信验证码
3  * @param user
4  * @return
5  */
6 @PostMapping("/sendMsg")
7 public R<String> sendMsg(@RequestBody User user, HttpSession session){
8     //获取手机号
9     String phone = user.getPhone();
10     if(StringUtils.isEmpty(phone)){
11         //生成随机的4位验证码
12         String code = ValidateCodeUtils.generateValidateCode(4).toString();
13         log.info("code={}",code);
14     }
```

```

15         //调用阿里云提供的短信服务API完成发送短信
16         //SMSUtils.sendMessage("瑞吉外卖","",phone,code);
17
18         //需要将生成的验证码保存到Session
19         session.setAttribute(phone,code);
20         return R.success("手机验证码短信发送成功");
21     }
22     return R.error("短信发送失败");
23 }

```

备注：

这里发送短信我们只需要调用封装的工具类中的方法即可，我们这个功能流程跑通，在测试中我们不用真正的发送短信，只需要将验证码信息，通过日志输出，登录时，我们直接从控制台就可以看到生成的验证码（实际上也就是发送到我们手机上的验证码）

5.4.2.3 验证码登录

在UserController中增加登录的方法 login，该方法的具体逻辑为：

- 1). 获取前端传递的手机号和验证码
- 2). 从Session中获取到手机号对应的正确的验证码
- 3). 进行验证码的比对，如果比对失败，直接返回错误信息
- 4). 如果比对成功，需要根据手机号查询当前用户，如果用户不存在，则自动注册一个新用户
- 5). 将登录用户的ID存储Session中

具体代码实现：

```

1  /**
2   * 移动端用户登录
3   * @param map
4   * @param session
5   * @return
6   */
7  @PostMapping("/login")
8  public R<User> login(@RequestBody Map map, HttpSession session){

```

```

9      log.info(map.toString());
10     //获取手机号
11     String phone = map.get("phone").toString();
12     //获取验证码
13     String code = map.get("code").toString();
14     //从Session中获取保存的验证码
15     Object codeInSession = session.getAttribute(phone);
16
17     //进行验证码的比对（页面提交的验证码和Session中保存的验证码比对）
18     if (codeInSession != null && codeInSession.equals(code)) {
19         //如果能够比对成功，说明登录成功
20
21         LambdaQueryWrapper<User> queryWrapper = new LambdaQueryWrapper<>();
22         queryWrapper.eq(User::getPhone, phone);
23
24         User user = userService.getOne(queryWrapper);
25         if (user == null) {
26             //判断当前手机号对应的用户是否为新用户，如果是新用户就自动完成注册
27             user = new User();
28             user.setPhone(phone);
29             user.setStatus(1);
30             userService.save(user);
31         }
32         session.setAttribute("user", user.getId());
33         return R.success(user);
34     }
35     return R.error("登录失败");
36 }

```

5.5 功能测试

代码完成后，重启服务，测试短信验证码的发送及登录功能。

1). 测试错误验证码的情况

```

c.i.reggie.controller.UserController      : {phone=13009091212, code=1111}
c.i.reggie.filter.LoginCheckFilter        : 拦截到请求: /user/sendMsg
c.i.reggie.filter.LoginCheckFilter        : 本次请求/user/sendMsg不需要处理
c.i.reggie.controller.UserController      : code=2193
        
```

输入错误验证码

2). 测试正确验证码的情况

```

c.i.reggie.controller.UserController      : {phone=13009091212, code=1111}
c.i.reggie.filter.LoginCheckFilter        : 拦截到请求: /user/sendMsg
c.i.reggie.filter.LoginCheckFilter        : 本次请求/user/sendMsg不需要处理
c.i.reggie.controller.UserController      : code=8343
        
```

输入正确验证码

登录成功进入首页

检查user表，用户的数据也插入进来了：

id	name	phone	sex	id_number	avatar	status
1424043150292332545	(Null)	13009091212	(Null)	(Null)	(Null)	1